



INSTITUTO PROFESIONAL
SAN SEBASTIAN

Depuración y optimización de algoritmos para análisis de datos

Sesión sincrónica

**INSTITUTO PROFESIONAL
ACREDITADO**



**GESTIÓN INSTITUCIONAL
DOCENCIA DE PREGRADO**
3 AÑOS / HASTA MARZO 2025



ACTIVIDADES

Apunte Activador

Objetivo de la unidad: asegurar correctitud, eficiencia y mantenibilidad del código analítico.

- Depurar con método: aislar causa raíz (breakpoints, inspección de variables, logging).
- Robustez: manejo de excepciones específicas y validación de entradas para evitar caídas.
- Medir antes de optimizar: profiling (cProfile, timeit) para ubicar cuellos de botella.
- Optimizar sin cambiar la semántica: vectorización (NumPy/pandas), groupby/agg, tipos eficientes.
- Pruebas de regresión: comparar “antes vs. después” con datasets sintéticos y tolerancias numéricas.
- Refactorización: funciones pequeñas, DRY, docstrings; separar cargar→limpiar→transformar→reportar.
- Cierre y trazabilidad: reporte breve con mejoras (tiempo/memoria), evidencia de exactitud y lecciones.

Mini-workflow recomendado: medir → corregir → probar → optimizar → validar → documentar.

Foro

La Dirección de Analítica de una institución chilena integra diariamente datos tabulares (millones de filas) para generar métricas operativas. El pipeline actual “funciona”, pero tiene problemas: tiempos de ejecución impredecibles, resultados inconsistentes cuando faltan archivos y dificultad para rastrear errores. Debe aplicar lo visto en la Unidad 3 (depuración y optimización de algoritmos): aislar causas raíz con *profiling* y *logging*, robustecer con manejo de excepciones, reemplazar bucles por vectorización (NumPy/pandas), y garantizar equivalencia de resultados con pruebas de regresión antes y después de optimizar. La meta: un flujo correcto, rápido y mantenible.

1. El *profiling* muestra que el 75% del tiempo se consume en un bucle que calcula una columna derivada fila a fila y que, además, hay peaks de errores cuando falta un archivo diario.
¿Qué acciones concretas propones para (a) reducir el tiempo y (b) evitar caídas por archivos faltantes? Justifica.
2. Tras optimizar con vectorización, ¿cómo demuestras que los resultados no cambiaron respecto de la versión previa? Describe un enfoque mínimo de pruebas.
3. El código original mezcla descarga, limpieza, transformación y reporte en un único notebook. **Propone una refactorización mínima** que facilite depurar, probar y optimizar en el futuro, sin cambiar la lógica.



Caso - Problema

Presentación del caso/problema

La Dirección de Transporte de la Región Metropolitana debe monitorear a diario el flujo de pasajeros por estación para detectar cambios abruptos, planificar refuerzos operativos y reportar tendencias semanales al comité de movilidad. Para ello, dispone de archivos CSV diarios con las columnas fecha, estación y entradas, que se consolidan cada fin de mes para calcular la variación porcentual diaria por estación y construir un ranking de crecimiento. El proceso actual fue armado rápidamente por un analista y “funciona”, pero es frágil: se cae si falta un archivo, mezcla estaciones al calcular variaciones y tarda demasiado cuando supera los 5 millones de registros.

Tu misión es optimizar ese proceso sin cambiar sus objetivos. En concreto, deberás (1) corregir el cálculo de variación diaria para que compare valores dentro de cada estación, (2) implementar manejo de excepciones y registro de incidentes para que el pipeline no se detenga ante archivos ausentes o vacíos, (3) reemplazar bucles ineficientes por operaciones vectorizadas con pandas/NumPy para generar el ranking en tiempos aceptables, y (4) diseñar pruebas de regresión que certifiquen que los resultados se mantienen tras las mejoras. Al finalizar, la Dirección contará con un flujo confiable, rápido y mantenible, listo para escalar y respaldar decisiones operativas con datos de calidad.

La Dirección de Transporte de la RM te entrega archivos diarios validaciones_YYYY-MM-DD.csv con estas columnas:

fecha (YYYY-MM-DD)

estacion (str)

entradas (int)

Presentación del caso/problema

Un analista hizo un script para:

1. Cargar todos los CSV de un mes.
2. Calcular la variación porcentual diaria de entradas por estación.
3. Generar un ranking de estaciones con mayor crecimiento promedio.

Problemas actuales:

1. Si falta un CSV de un día, el script se cae.
2. La variación se calcula así:
 - `df['prev'] = df.sort_values('fecha')['entradas'].shift(1)`
 - `df['var_pct'] = (df['entradas'] - df['prev']) / df['prev'] * 100`
3. Resultado: variaciones absurdas (mezcla estaciones distintas).
4. El ranking se calcula con un bucle fila a fila (muy lento en 5M de registros).

No hay pruebas automatizadas; los cambios recientes podrían romper el resultado.

PREGUNTAS

1. Detecta el error en el cálculo de la variación diaria y corrígelo.
2. Implementa manejo de excepciones para que el pipeline no se caiga si falta un CSV y registre el incidente.
3. Reemplaza el bucle que calcula el ranking por una operación vectorizada eficiente.
4. Diseña 3 pruebas de regresión para garantizar que la optimización no cambió los resultados.



INSTITUTO PROFESIONAL
SAN SEBASTIAN

Depuración y optimización de algoritmos para análisis de datos

Sesión sincrónica

**INSTITUTO PROFESIONAL
ACREDITADO**



**GESTIÓN INSTITUCIONAL
DOCENCIA DE PREGRADO**
3 AÑOS / HASTA MARZO 2025